

1. Create VPC with 2 private and 2 public subnets.

Steps

1. Log in to the AWS Management Console.
2. Search for and open VPC.
3. Select Your VPCs from the left navigation menu.
4. Click Create VPC.
5. Select VPC only.
6. Enter the following details:
 - a. Name tag: DevOps-VPC
 - b. IPv4 CIDR block: 10.0.0.0/16
7. Click Create VPC.
8. After creating the VPC, go to Subnets → Create subnet.
9. Select the VPC:
 - a. VPC: DevOps-VPC
10. Create four subnets with unique CIDR blocks:

Subnet Name	CIDR Block	Type
Public-Subnet-1	10.0.5.0/24	Public
Public-Subnet-2	10.0.6.0/24	Public
Private-Subnet-1	10.0.7.0/24	Private
Private-Subnet-2	10.0.8.0/24	Private

11. Select an available Availability Zone for each subnet, or leave it as No preference.
12. Click Create subnet.
13. Verify that all four subnets are created under DevOps-VPC.

aws

Search

[Alt+S] Ask Amazon Q

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik Asif

VPC > Your VPCs > Create VPC

1

Create VPC Info

A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances.

VPC settings

Resources to create Info

Create only the VPC resource or the VPC and other networking resources.

☒ VPC only

☐ VPC and more

Name tag - optional

Creates a tag with a key of 'Name' and a value that you specify.

DevOps-VPC

IPv4 CIDR block Info

☒ IPv4 CIDR manual input

☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

10.0.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block Info

☒ No IPv6 CIDR block

☐ IPAM-allocated IPv6 CIDR block

☐ Amazon-provided IPv6 CIDR block

☐ IPv6 CIDR owned by me

aws

Search

[Alt+S] Ask Amazon Q

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik Asif

VPC > Your VPCs > vpc-02e579ff687daa21e

1

VPC dashboard <

Filter by VPC

AWS Global View

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

You successfully created vpc-02e579ff687daa21e / DevOps-VPC

vpc-02e579ff687daa21e / DevOps-VPC

Actions

Details Info

VPC ID

vpc-02e579ff687daa21e

State

Available

DNS resolution

Enabled

Tenancy

default

Main network ACL

acl-0711af7f8857b5551

Default VPC

No

IPv6 CIDR (Network border group)

-

Network Address Usage metrics

Disabled

Encryption control ID

-

Encryption control mode

-

Block Public Access

Off

DHCP option set

dopt-07b465a3b51c33fcd

IPv4 CIDR

10.0.0.0/16

Route 53 Resolver DNS Firewall rule groups

-

DNS hostnames

Disabled

Main route table

rtb-Obaec1787444bccccd

IPv6 pool

-

Owner ID

331116047691

Resource map CIDRs Flow logs Tags Related resources Integrations

Resource map Info

Show all details

VPC

Subnets (0)

Route tables (1)

CloudShell

Agent Toolkit for AWS

Feedback

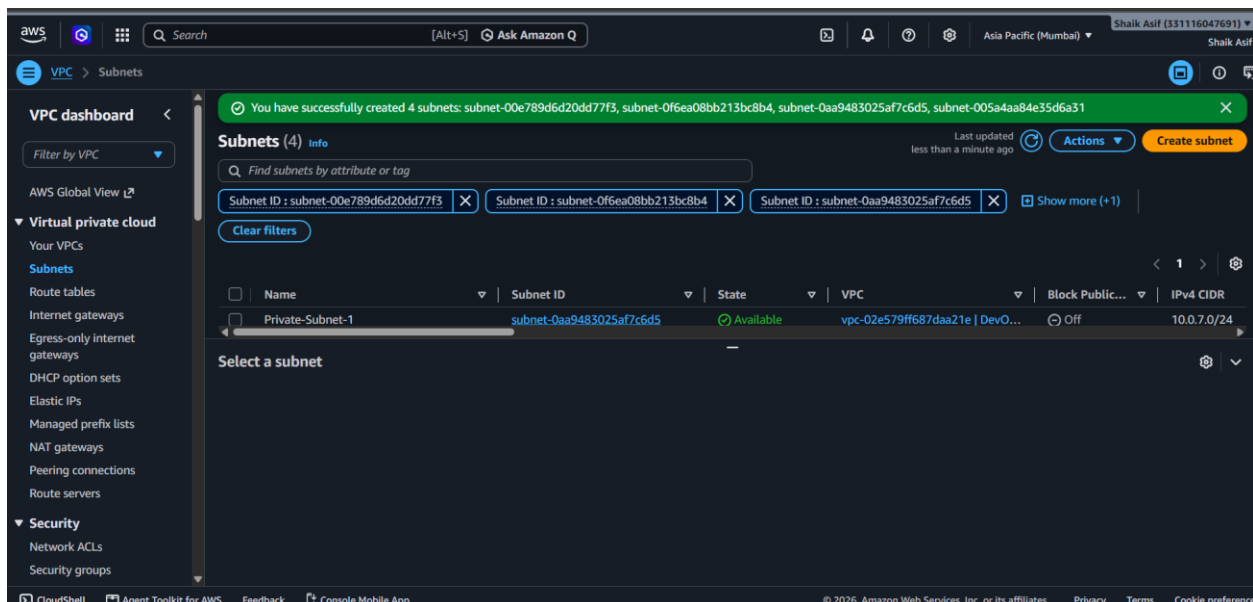
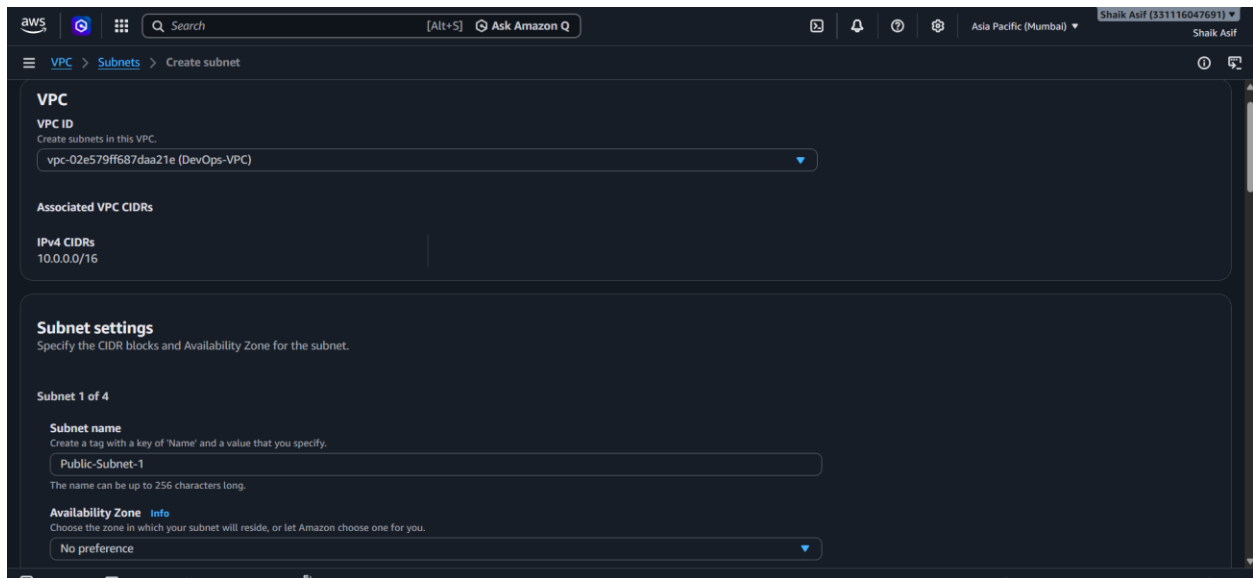
Console Mobile App

© 2026 Amazon Web Services, Inc. or its affiliates

Privacy

Terms

Cookie preferences



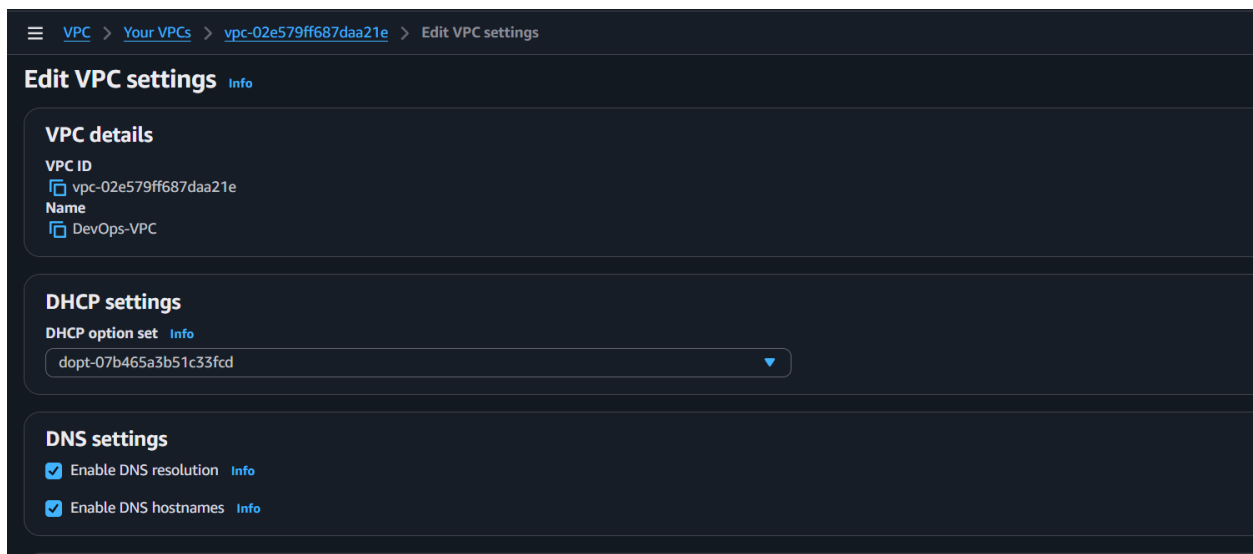
Result

VPC was successfully created with CIDR block **10.0.0.0/16**, along with two public subnets and two private subnets. The four subnets were successfully associated with the newly created VPC.

2. Enable DNS Hostname in VPC.

Steps

1. Log in to the AWS Management Console.
2. Search for VPC and open the VPC Dashboard.
3. From the left navigation menu, select Your VPCs.
4. Select your VPC:
 - a. Name: DevOps-VPC
 - b. VPC ID: vpc-02e579ff687daa21e
5. Click Actions.
6. Select Edit VPC settings.
7. Under DNS settings, verify:
 - a. Enable DNS resolution → ☒ Enabled
 - b. Enable DNS hostnames → ☒ Enabled
8. Click Save changes.



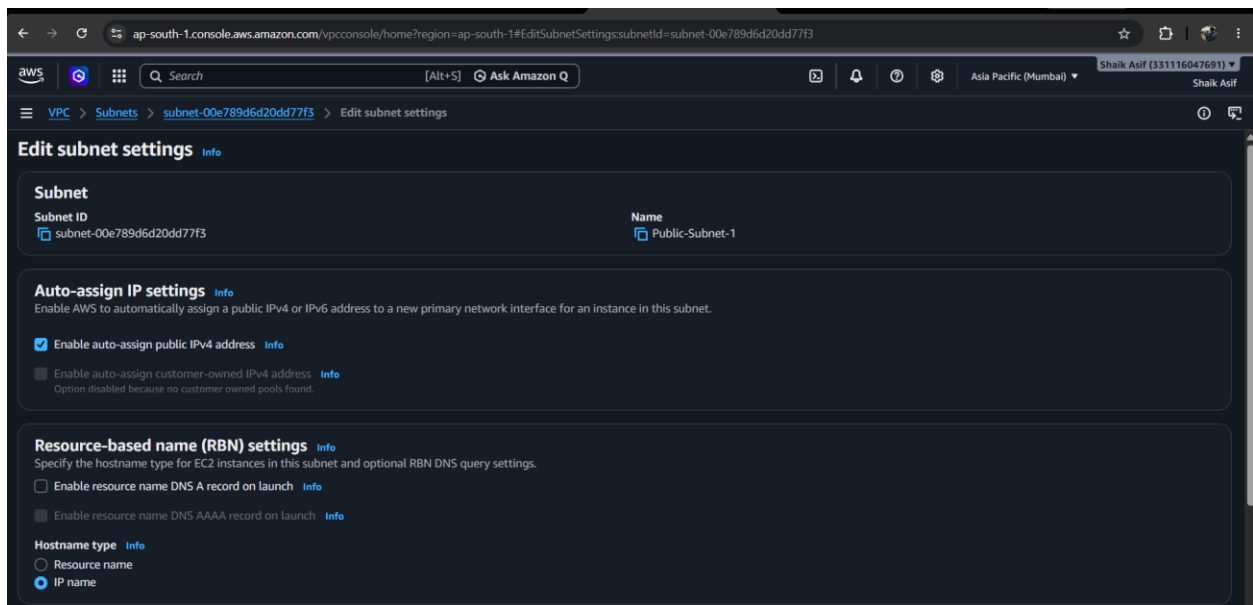
Result

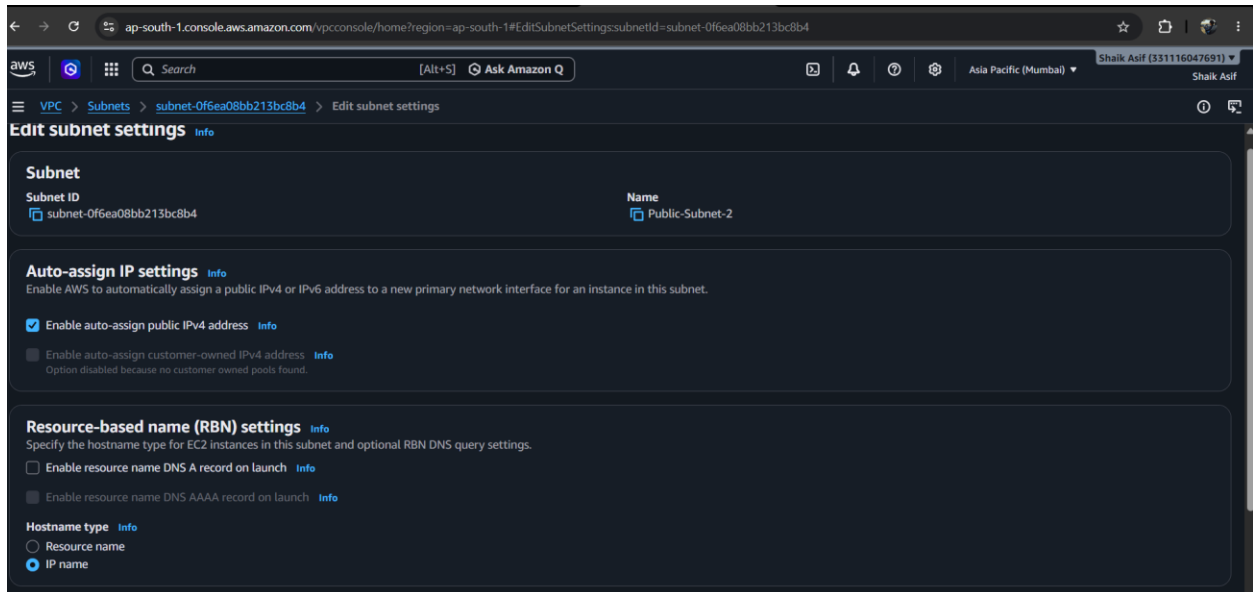
DNS Hostnames was successfully enabled for the DevOps-VPC. DNS resolution was also verified as enabled, allowing resources in the VPC to receive DNS hostnames.

3. Enable Auto Assign Public IP in 2 Public Subnets

Steps

1. Log in to the AWS Management Console.
2. Search for VPC and open the VPC Dashboard.
3. From the left navigation menu, select Subnets.
4. Select Public-Subnet-1.
5. Click Actions → Edit subnet settings.
6. Under Auto-assign IP settings, verify:
 - a. Enable auto-assign public IPv4 address →  Enabled
7. Click Save changes.
8. Repeat the same steps for Public-Subnet-2.
9. Verify that Enable auto-assign public IPv4 address is enabled for both public subnets.





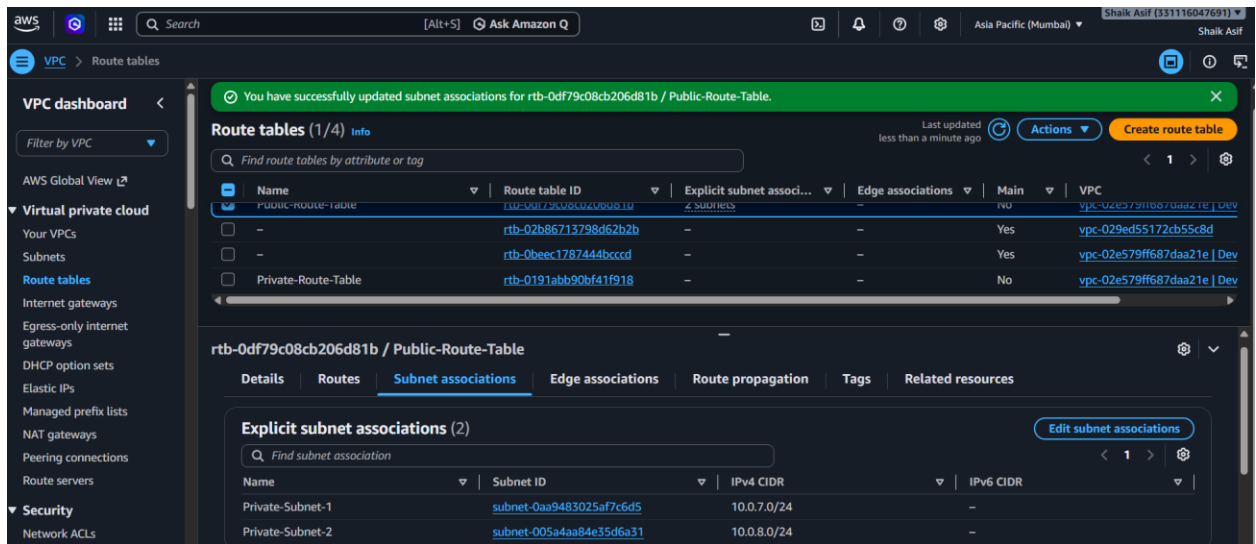
Result

Auto-assign Public IPv4 address was successfully enabled for both Public-Subnet-1 and Public-Subnet-2. EC2 instances launched in these subnets can automatically receive a public IPv4 address.

4. Add 2 Private Subnets in Private Route Table

Steps

1. Log in to the **AWS Management Console**.
2. Search for **VPC** and open the **VPC Dashboard**.
3. From the left navigation menu, select **Route Tables**.
4. Select the **Private-Route-Table** that you created.
5. At the bottom of the page, select the **Subnet associations** tab.
6. Click **Edit subnet associations**.
7. Select the following two subnets:
 - a. ☒ **Private-Subnet-1**
 - b. ☒ **Private-Subnet-2**
8. Click **Save associations**.



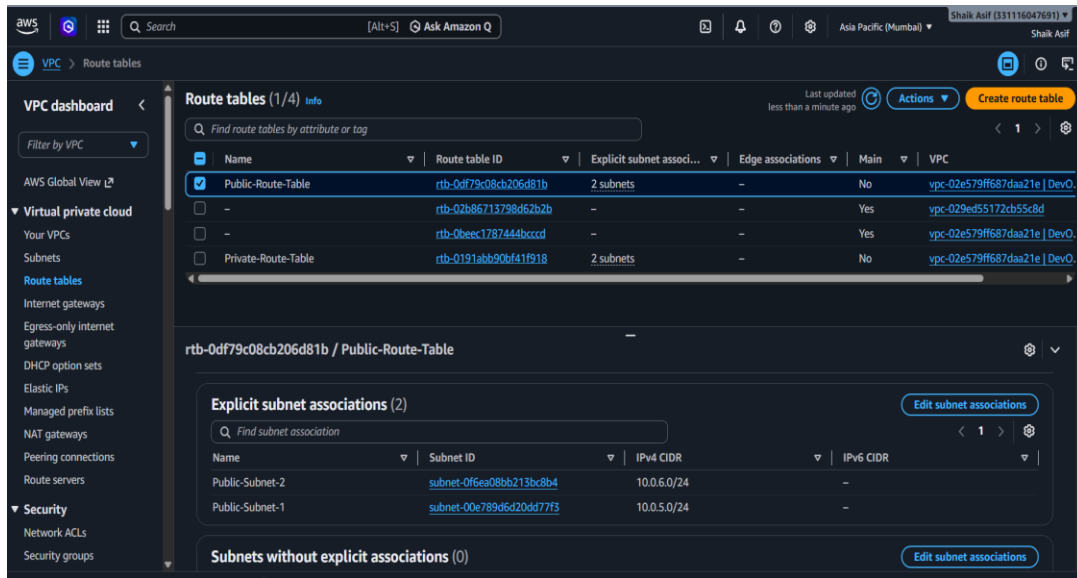
Result

Two private subnets, **Private-Subnet-1** and **Private-Subnet-2**, were successfully associated with the **Private Route Table**.

5. Add 2 public subnets in public route table.

Steps

1. Log in to the **AWS Management Console**.
2. Search for **VPC** and open the **VPC Dashboard**.
3. From the left navigation menu, select **Route Tables**.
4. Select the **Public-Route-Table** that you created.
5. At the bottom of the page, select the **Subnet associations** tab.
6. Click **Edit subnet associations**.
7. Select the following two subnets:
 - a. ☒ **Public-Subnet-1**
 - b. ☒ **Public-Subnet-2**
8. Click **Save associations**.



Result

Two public subnets, **Public-Subnet-1** and **Public-Subnet-2**, were successfully associated with the Public Route Table.

6. Public route table will have the routes to internet and local.

Steps

1. Log in to the AWS Management Console.
2. Search for VPC and open the VPC Dashboard.
3. From the left navigation menu, select Route Tables.
4. Select Public-Route-Table.
5. Click the Routes tab at the bottom.

Check the Existing Local Route

You should already see:

Destination	Target
10.0.0.0/16	local

This route is automatically created by AWS when the route table is created.

Add Internet Route

6. Click Edit routes.
7. Click Add route.
8. Enter:

Destination:

0.0.0.0/0

Target:

- Select Internet Gateway
 - Choose DevOps-IGW
9. Click Save changes.

VPC dashboard

Filter by VPC

AWS Global View

Virtual private cloud

- Your VPCs
- Subnets
- Route tables**
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

Security

- Network ACLs
- Security groups

Updated routes for rtb-0df79c08cb206d81b / Public-Route-Table successfully

rtb-0df79c08cb206d81b / Public-Route-Table

Details

Route table ID: rtb-0df79c08cb206d81b

VPC: vpc-02e579ff687daa21e | DevOps-VPC

Main: No

Owner ID: 331116047691

Explicit subnet associations: 2 subnets

Edge associations: -

Routes

Subnet associations | Edge associations | Route propagation | Tags | Related resources

Routes (2)

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-0ad69458c66ebc6a1	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

Result

The Public Route Table was successfully configured with a local route (10.0.0.0/16) and an internet route (0.0.0.0/0) through the Internet Gateway. This enables internet connectivity for resources in the associated public subnets

7..Create EC2 in public subnet with t2.micro and install PHP.

Steps

Part 1: Launch EC2 Instance

1. Log in to the AWS Management Console.
2. Search for EC2 and open the EC2 Dashboard.
3. Select Instances → Launch instances.
4. Enter the following details:

Name:

Public-PHP-Server

AMI:

- Select Amazon Linux 2023.

Instance type:

t2.micro

Key pair:

- Select your existing key pair.
- If you don't have one, create and download a new key pair.

Part 2: Configure Network

5. Under Network settings, click Edit.
6. Select:
 - a. VPC: DevOps-VPC
 - b. Subnet: Public-Subnet-1
 - c. Auto-assign Public IP: Enable

7. Under Security Group, create/select a security group.

Allow:

Type	Port	Source
SSH	22	My IP
HTTP	80	Anywhere (0.0.0.0/0)

8. Click Launch instance.

The screenshot shows the AWS Management Console interface for an EC2 instance. The left sidebar contains navigation links for EC2, Dashboard, AWS Global View, Events, Instances, Instance Types, Launch Templates, Spot Requests, Savings Plans, Reserved Instances, Dedicated Hosts, Capacity Reservations, Capacity Manager, Images, AMIs, AMI Catalog, Elastic Block Store, Volumes, and Snapshots. The main content area displays the 'Instance summary' for instance ID i-019bbcd54e7448398, which is a Public-PHP-Server. The instance is in a 'Running' state. Key details include: Public IPv4 address 13.203.228.215, Private IPv4 address 10.0.5.97, Public DNS ec2-13-203-228-215.ap-south-1.compute.amazonaws.com, Instance type t3.micro, VPC ID vpc-02e579ff687daa21e, Subnet ID subnet-00e789d6d20dd77f3, and Instance ARN arn:aws:ec2:ap-south-1:33116047691:instance/i-019bbcd54e7448398. The instance is associated with the IAM role 'Required' and the IMDSv2 feature is set to 'Required'.

The screenshot shows the 'Connect to Linux instance' page in the AWS Management Console. The page provides instructions on how to connect to the instance i-019bbcd54e7448398, which is in a 'Running' state. It offers two main connection methods: 'In web browser' and 'In SSH client'. Under 'In web browser', there are four options: 'EC2 Instance Connect' (selected), 'EC2 Instance Connect Endpoint', 'SSM Session Manager', and 'EC2 Serial Console'. Each option has a brief description of its use case. The 'Settings' section at the bottom allows choosing the public IP address (IPv4 or IPv6) and provides a 'Connect' button.

Result:

8 Configure NAT gateway in public subnet and connect to private instance.

Go to:

Select your existing VPC:

Create:

Click Create subnet.

Your existing public subnet is:

10.0.5.0/24

So 10.0.6.0/24 is a separate subnet.

Step 2: Create an Elastic IP

Go to:

VPC → Elastic IPs → Allocate Elastic IP address

Click:

Allocate

You should get an Elastic IP.

Step 3: Create NAT Gateway

Go to:

VPC → NAT gateways → Create NAT gateway

Enter:

Setting	Value
Name	DevOps-NAT-Gateway
Subnet	Public-Subnet-1
Connectivity type	Public
Elastic IP allocation ID	Select the Elastic IP you created

Click:

Create NAT gateway

Wait until the status becomes:

Available

Very important

The NAT Gateway must be placed in your PUBLIC subnet, not the private subnet.

Step 4: Create a Private Route Table

Go to:

VPC → Route tables → Create route table

Use:

Name: Private-Route-Table

VPC: DevOps-VPC

Click Create route table.

Step 5: Add NAT route

Open:

Private-Route-Table → Routes → Edit routes

Add:

Destination: 0.0.0.0/0

Target: NAT Gateway

Select:

DevOps-NAT-Gateway

Save changes.

You should now have:

10.0.0.0/16 → local

0.0.0.0/0 → nat-xxxxxxx

Step 6: Associate Private Route Table with Private Subnet

Inside:

Private-Route-Table → Subnet associations

Click:

Edit subnet associations

Select:

Private-Subnet-1

Click:

Save associations

Now your routing is:

Private EC2

|

Private Subnet

|

Private Route Table

|

NAT Gateway

|

Public Subnet

|

Internet Gateway

|

Internet

Step 7: Launch the Private EC2

Go to:

EC2 → Launch instance

Use:

Name: Private-Server

AMI: Amazon Linux 2023

Instance type: t3.micro

Key pair: shaik123

Under Network settings, select:

VPC: DevOps-VPC

Subnet: Private-Subnet-1

Auto-assign public IP: Disable

This is important.

The private instance should have:

Public IPv4: None

Private IPv4: 10.0.6.x

Step 8: Configure Security Group

For the private instance, create/use a security group such as:

Private-Server-SG

Inbound rule:

Type: SSH

Port: 22

Source: Public-Server-SG

Do not use:

0.0.0.0/0

for SSH on the private server.

Your public server security group can continue allowing SSH from your IP.

Step 9: Connect to the Public Server

You already successfully did this:

```
ssh -i "shaik123.pem" ec2-user@13.203.228.215
```

You should see:

```
[ec2-user@ip-10-0-5-97 ~]$
```

That's your public EC2.

Step 10: Connect from Public EC2 to Private EC2

First find the private IP address of your new private EC2.

For example:

10.0.6.25

Easiest beginner method

You can copy the PEM file to the public server, then connect:

```
chmod 400 shaik123.pem  
ssh -i shaik123.pem ec2-user@10.0.6.25
```

You should then get:

```
[ec2-user@ip-10-0-6-25 ~]$
```

That means you successfully connected to the private instance.

For a real production environment, don't copy private keys onto bastion servers. SSH agent forwarding or AWS Systems Manager Session Manager is safer.

Step 11: Test NAT Gateway

Once you are inside the private EC2, run:

```
curl https://www.google.com
```

or:

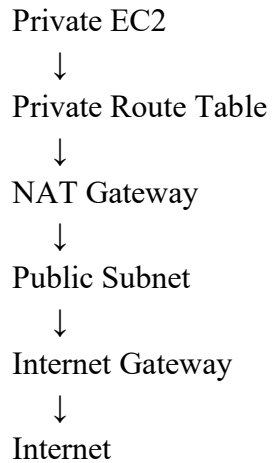
```
ping -c 4 8.8.8.8
```

You can also test package access:

```
sudo dnf update -y
```

If these work, your private instance is reaching the internet through the NAT Gateway.

Check the path:



Expected final result

You should have:

Resource	Configuration
Public subnet	10.0.5.0/24
Private subnet	10.0.6.0/24
Public EC2	t3.micro, public IP
Private EC2	t3.micro, no public IP
Internet Gateway	Attached to VPC
NAT Gateway	Located in public subnet
Elastic IP	Attached to NAT Gateway
Public route table	0.0.0.0/0 → Internet Gateway
Private route table	0.0.0.0/0 → NAT Gateway

aws [Search] [Alt+S] Asia Pacific (Mumbai) Shaik Asif (331116047691) Shaik Asif

VPC > NAT gateways

NAT gateways (1) info

Find NAT gateways by attribute or tag

Name	NAT gateway ID	Connectivity...	State	State message	Availability ...	Route table ID	P
DevOps-NAT-Gateway	nat-1db8b11f38891bcf5	Public	Available	-	Regional	rtb-0a5020142...	1

Select a NAT gateway

VPC dashboard

- Virtual private cloud
 - Your VPCs
 - Subnets
 - Route tables
 - Internet gateways
 - Egress-only internet gateways
 - DHCP option sets
 - Elastic IPs
 - Managed prefix lists
 - NAT gateways**
 - Peering connections
 - Route servers
- Security
 - Network ACLs
 - Security groups

VPC > Route tables > rtb-00a5d199b298c8cfc

Route table rtb-00a5d199b298c8cfc / Private-Route-Table was created successfully.

rtb-00a5d199b298c8cfc / Private-Route-Table

Actions

Details info

Route table ID
rtb-00a5d199b298c8cfc

VPC
vpc-02e579ff687daa21e | DevOps-VPC

Main
No

Owner ID
331116047691

Explicit subnet associations
-

Edge associations
-

Routes Subnet associations Edge associations Route propagation Tags Related resources

Routes (1)

Filter routes

Destination	Target	Status	Propagated	Route Origin
10.0.0.0/16	local	Active	No	Create Route Table

aws [Search] [Alt+S] Asia Pacific (Mumbai) Shaik Asif (331116047691) Shaik Asif

VPC > Route tables > rtb-00a5d199b298c8cfc

Updated routes for rtb-00a5d199b298c8cfc / Private-Route-Table successfully

rtb-00a5d199b298c8cfc / Private-Route-Table

Actions

Details info

Route table ID
rtb-00a5d199b298c8cfc

VPC
vpc-02e579ff687daa21e | DevOps-VPC

Main
No

Owner ID
331116047691

Explicit subnet associations
-

Edge associations
-

Routes Subnet associations Edge associations Route propagation Tags Related resources

Routes (2)

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-1db8b11f38891bcf5	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

Step 1 — Connect to Private EC2

From your public EC2, connect to the private EC2:

```
ssh -i shaik123.pem ec2-user@10.0.9.174
```

Expected:

```
[ec2-user@ip-10-0-9-174 ~]$
```

Step 2 — Install Java 17

```
sudo dnf install java-17-amazon-corretto -y
```

Verify:

```
java -version
```

Expected: Java 17 should be displayed.

 **Screenshot:** java -version

Step 3 — Download Tomcat

Go to /tmp:

```
cd /tmp
```

Download Tomcat 10.1.59:

```
wget https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.59/bin/apache-tomcat-10.1.59.tar.gz
```

Step 4 — Extract Tomcat

```
sudo tar -xzf apache-tomcat-10.1.59.tar.gz -C /opt/
```

Rename the directory:

```
sudo mv /opt/apache-tomcat-10.1.59 /opt/tomcat
```

Step 5 — Create Tomcat User

```
sudo useradd -r -m -U -d /opt/tomcat -s /bin/false tomcat
```

You may receive this warning:

```
useradd: warning: the home directory /opt/tomcat already exists.
```

This is okay because /opt/tomcat already exists.

Set ownership:

```
sudo chown -R tomcat:tomcat /opt/tomcat
```

Step 6 — Verify Tomcat Files

```
sudo ls -la /opt/tomcat/bin
```

You should see:

```
startup.sh
shutdown.sh
catalina.sh
```

The .sh files should have executable permission.

Step 7 — Create Tomcat Service

```
sudo vi /etc/systemd/system/tomcat.service
```

Press **i** and add:

```
[Unit]
Description=Apache Tomcat
After=network.target
```

```
[Service]
Type=forking
```

User=tomcat
Group=tomcat

Environment="JAVA_HOME=/usr/lib/jvm/java-17-amazon-corretto"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"

ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh

Restart=on-failure

[Install]
WantedBy=multi-user.target

Save:

Esc
:wq
Enter

Step 8 — Start Tomcat

Run:

```
sudo systemctl daemon-reload  
sudo systemctl enable tomcat  
sudo systemctl start tomcat
```

Check status:

```
sudo systemctl status tomcat
```

Expected:

Active: active (running)

 **Screenshot:** Tomcat status.

Step 9 — Verify Tomcat

Check port 8080:

```
sudo ss -lntp | grep 8080
```

Test Tomcat:

```
curl http://localhost:8080
```

Tomcat HTML output should be displayed.

 **Screenshot:** curl <http://localhost:8080>

Step 10 — Deploy Sample Application

Create the application directory:

```
sudo mkdir -p /opt/tomcat/webapps/sample
```

Create the HTML file:

```
sudo vi /opt/tomcat/webapps/sample/index.html
```

Add:

```
<html>
<head>
  <title>Sample Application</title>
</head>
<body>
  <h1>Welcome to My Sample Application</h1>
  <p>Tomcat is successfully installed on the private EC2.</p>
</body>
</html>
```

Save:

Esc

:wq

Enter

Set ownership:

sudo chown -R tomcat:tomcat /opt/tomcat/webapps/sample

Step 11 — Test Sample Application

Run:

curl <http://localhost:8080/sample/>

Expected output will contain:

<h1>Welcome to My Sample Application</h1>

```
ec2-user@ip-10-0-9-174:tmp
[ec2-user@ip-10-0-9-174 ~]$ sudo dnf install java-17-amazon-corretto -y
Amazon Linux 2023 repository
Amazon Linux 2023 kernel Livepatch repository
Dependencies resolved.

Package Architecture Version Repository Size
-----
Installing:
java-17-amazon-corretto x86_64 1:17.0.20-8-1.amzn2023.1 amazonlinux 220 k
Installing dependencies:
alsa-lib x86_64 1.2.7-2.1.amzn2023.0.3 amazonlinux 503 k
cairo x86_64 1.18.0-4.amzn2023.0.3 amazonlinux 717 k
dejavu-sans-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.3 M
dejavu-sans-mono-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 467 k
dejavu-serif-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.0 M
fontconfig x86_64 2.13.94-2.amzn2023.0.2 amazonlinux 273 k
fontconfig-filesystem noarch 1:12.0.5-12.amzn2023.0.2 amazonlinux 9.5 k
freetype x86_64 2.13.2-5.amzn2023.0.2 amazonlinux 422 k
glib2 x86_64 5.2.1-9.amzn2023.0.4 amazonlinux 49 k
google-noto-fonts-common noarch 20240401-1.amzn2023.0.2 amazonlinux 17 k
google-noto-sans-vf-fonts noarch 20240401-1.amzn2023.0.2 amazonlinux 593 k
graphite2 x86_64 1.3.14-7.amzn2023.0.3 amazonlinux 97 k
harfbuzz x86_64 7.0.0-2.amzn2023.0.2 amazonlinux 873 k
java-17-amazon-corretto-headless x86_64 1:17.0.20-8-1.amzn2023.1 amazonlinux 91 M
javapackages-filesystem noarch 6.0.0-7.amzn2023.0.6 amazonlinux 12 k
langpacks-core-font-en noarch 3.0-21.amzn2023.0.4 amazonlinux 10 k
libICE x86_64 1.1.1-3.amzn2023.0.1 amazonlinux 76 k
libXt x86_64 1.2.4-3.amzn2023.0.1 amazonlinux 45 k
libXt1 x86_64 1.8.10-2.amzn2023.0.1 amazonlinux 659 k
libXt1-common noarch 1.8.10-2.amzn2023.0.1 amazonlinux 147 k
libXau x86_64 1.0.11-6.amzn2023.0.1 amazonlinux 33 k
libXext x86_64 1.3.6-1.amzn2023.0.1 amazonlinux 42 k
libXft x86_64 1.8.2-1.amzn2023.0.1 amazonlinux 42 k
libXinerama x86_64 1.1.5-6.amzn2023.0.1 amazonlinux 16 k
libXrandr x86_64 1.5.4-3.amzn2023.0.1 amazonlinux 29 k
libXrender x86_64 0.9.11-6.amzn2023.0.1 amazonlinux 29 k
libXt x86_64 1.3.0-3.amzn2023.0.1 amazonlinux 183 k
libXtst x86_64 1.2.5-1.amzn2023.0.1 amazonlinux 22 k
libbrotli x86_64 1.0.9-4.amzn2023.0.2 amazonlinux 315 k
libjpeg-turbo x86_64 2.1.4-2.amzn2023.0.5 amazonlinux 190 k
libpng x86_64 2:1.6.37-10.amzn2023.0.13 amazonlinux 128 k
libxcb x86_64 1.17.0-1.amzn2023.0.1 amazonlinux 235 k
pixman x86_64 0.43.4-1.amzn2023.0.4 amazonlinux 296 k
xml-common noarch 0.6.3-56.amzn2023.0.2 amazonlinux 32 k

Transaction Summary
-----
Install 35 Packages
Total download size: 100 M
Installed size: 262 M
Downloading Packages:
(1/35): alsa-lib-1.2.7-2.1.amzn2023.0.3.x86_64.rpm 13 MB/s | 503 kB 00:00
(2/35): cairo-1.18.0-4.amzn2023.0.3.x86_64.rpm 15 MB/s | 717 kB 00:00
(3/35): dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch.rpm 21 MB/s | 1.3 MB 00:00
(4/35): dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch.rpm 15 MB/s | 467 kB 00:00
(5/35): dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch.rpm 15 MB/s | 1.0 MB 00:00
(6/35): fontconfig-2.13.94-2.amzn2023.0.2.x86_64.rpm 15 MB/s | 273 kB 00:00
(7/35): fontconfig-filesystem-1:12.0.5-12.amzn2023.0.2.noarch.rpm 15 MB/s | 9.5 kB 00:00
(8/35): freetype-2.13.2-5.amzn2023.0.2.x86_64.rpm 15 MB/s | 422 kB 00:00
(9/35): glib2-5.2.1-9.amzn2023.0.4.x86_64.rpm 15 MB/s | 49 kB 00:00
(10/35): google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch.rpm 15 MB/s | 17 kB 00:00
(11/35): google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch.rpm 15 MB/s | 593 kB 00:00
(12/35): graphite2-1.3.14-7.amzn2023.0.3.x86_64.rpm 15 MB/s | 97 kB 00:00
(13/35): harfbuzz-7.0.0-2.amzn2023.0.2.x86_64.rpm 15 MB/s | 873 kB 00:00
(14/35): java-17-amazon-corretto-headless-1:17.0.20-8-1.amzn2023.1.x86_64.rpm 15 MB/s | 91 MB 00:00
(15/35): javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch.rpm 15 MB/s | 12 kB 00:00
(16/35): langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch.rpm 15 MB/s | 10 kB 00:00
(17/35): libICE-1.1.1-3.amzn2023.0.1.x86_64.rpm 15 MB/s | 76 kB 00:00
(18/35): libXt-1.2.4-3.amzn2023.0.1.x86_64.rpm 15 MB/s | 45 kB 00:00
(19/35): libXt1-1.8.10-2.amzn2023.0.1.x86_64.rpm 15 MB/s | 659 kB 00:00
(20/35): libXt1-common-1.8.10-2.amzn2023.0.1.noarch.rpm 15 MB/s | 147 kB 00:00
(21/35): libXau-1.0.11-6.amzn2023.0.1.x86_64.rpm 15 MB/s | 33 kB 00:00
(22/35): libXext-1.3.6-1.amzn2023.0.1.x86_64.rpm 15 MB/s | 42 kB 00:00
(23/35): libXft-1.8.2-1.amzn2023.0.1.x86_64.rpm 15 MB/s | 42 kB 00:00
(24/35): libXinerama-1.1.5-6.amzn2023.0.1.x86_64.rpm 15 MB/s | 16 kB 00:00
(25/35): libXrandr-1.5.4-3.amzn2023.0.1.x86_64.rpm 15 MB/s | 29 kB 00:00
(26/35): libXrender-0.9.11-6.amzn2023.0.1.x86_64.rpm 15 MB/s | 29 kB 00:00
(27/35): libXt-1.3.0-3.amzn2023.0.1.x86_64.rpm 15 MB/s | 183 kB 00:00
(28/35): libXtst-1.2.5-1.amzn2023.0.1.x86_64.rpm 15 MB/s | 22 kB 00:00
(29/35): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm 15 MB/s | 315 kB 00:00
(30/35): libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64.rpm 15 MB/s | 190 kB 00:00
(31/35): libpng-2:1.6.37-10.amzn2023.0.13.x86_64.rpm 15 MB/s | 128 kB 00:00
(32/35): libxcb-1.17.0-1.amzn2023.0.1.x86_64.rpm 15 MB/s | 235 kB 00:00
(33/35): pixman-0.43.4-1.amzn2023.0.4.x86_64.rpm 15 MB/s | 296 kB 00:00
(34/35): xml-common-0.6.3-56.amzn2023.0.2.noarch.rpm 15 MB/s | 32 kB 00:00
(35/35): java-17-amazon-corretto-1:17.0.20-8-1.amzn2023.1.x86_64.rpm 15 MB/s | 220 kB 00:00
Total: 100 MB/s | 100 MB 00:00
Installed: 35 Packages
Total size: 262 MB
Installed size: 262 MB
Downloading Packages:
(1/35): alsa-lib-1.2.7-2.1.amzn2023.0.3.x86_64.rpm 13 MB/s | 503 kB 00:00
(2/35): cairo-1.18.0-4.amzn2023.0.3.x86_64.rpm 15 MB/s | 717 kB 00:00
(3/35): dejavu-sans-fonts-2.37-16.amzn2023.0.2.noarch.rpm 21 MB/s | 1.3 MB 00:00
(4/35): dejavu-sans-mono-fonts-2.37-16.amzn2023.0.2.noarch.rpm 15 MB/s | 467 kB 00:00
(5/35): dejavu-serif-fonts-2.37-16.amzn2023.0.2.noarch.rpm 15 MB/s | 1.0 MB 00:00
(6/35): fontconfig-2.13.94-2.amzn2023.0.2.x86_64.rpm 15 MB/s | 273 kB 00:00
(7/35): fontconfig-filesystem-1:12.0.5-12.amzn2023.0.2.noarch.rpm 15 MB/s | 9.5 kB 00:00
(8/35): freetype-2.13.2-5.amzn2023.0.2.x86_64.rpm 15 MB/s | 422 kB 00:00
(9/35): glib2-5.2.1-9.amzn2023.0.4.x86_64.rpm 15 MB/s | 49 kB 00:00
(10/35): google-noto-fonts-common-20240401-1.amzn2023.0.2.noarch.rpm 15 MB/s | 17 kB 00:00
(11/35): google-noto-sans-vf-fonts-20240401-1.amzn2023.0.2.noarch.rpm 15 MB/s | 593 kB 00:00
(12/35): graphite2-1.3.14-7.amzn2023.0.3.x86_64.rpm 15 MB/s | 97 kB 00:00
(13/35): harfbuzz-7.0.0-2.amzn2023.0.2.x86_64.rpm 15 MB/s | 873 kB 00:00
(14/35): java-17-amazon-corretto-headless-1:17.0.20-8-1.amzn2023.1.x86_64.rpm 15 MB/s | 91 MB 00:00
(15/35): javapackages-filesystem-6.0.0-7.amzn2023.0.6.noarch.rpm 15 MB/s | 12 kB 00:00
(16/35): langpacks-core-font-en-3.0-21.amzn2023.0.4.noarch.rpm 15 MB/s | 10 kB 00:00
(17/35): libICE-1.1.1-3.amzn2023.0.1.x86_64.rpm 15 MB/s | 76 kB 00:00
(18/35): libXt-1.2.4-3.amzn2023.0.1.x86_64.rpm 15 MB/s | 45 kB 00:00
(19/35): libXt1-1.8.10-2.amzn2023.0.1.x86_64.rpm 15 MB/s | 659 kB 00:00
(20/35): libXt1-common-1.8.10-2.amzn2023.0.1.noarch.rpm 15 MB/s | 147 kB 00:00
(21/35): libXau-1.0.11-6.amzn2023.0.1.x86_64.rpm 15 MB/s | 33 kB 00:00
(22/35): libXext-1.3.6-1.amzn2023.0.1.x86_64.rpm 15 MB/s | 42 kB 00:00
(23/35): libXft-1.8.2-1.amzn2023.0.1.x86_64.rpm 15 MB/s | 42 kB 00:00
(24/35): libXinerama-1.1.5-6.amzn2023.0.1.x86_64.rpm 15 MB/s | 16 kB 00:00
(25/35): libXrandr-1.5.4-3.amzn2023.0.1.x86_64.rpm 15 MB/s | 29 kB 00:00
(26/35): libXrender-0.9.11-6.amzn2023.0.1.x86_64.rpm 15 MB/s | 29 kB 00:00
(27/35): libXt-1.3.0-3.amzn2023.0.1.x86_64.rpm 15 MB/s | 183 kB 00:00
(28/35): libXtst-1.2.5-1.amzn2023.0.1.x86_64.rpm 15 MB/s | 22 kB 00:00
(29/35): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm 15 MB/s | 315 kB 00:00
(30/35): libjpeg-turbo-2.1.4-2.amzn2023.0.5.x86_64.rpm 15 MB/s | 190 kB 00:00
(31/35): libpng-2:1.6.37-10.amzn2023.0.13.x86_64.rpm 15 MB/s | 128 kB 00:00
(32/35): libxcb-1.17.0-1.amzn2023.0.1.x86_64.rpm 15 MB/s | 235 kB 00:00
(33/35): pixman-0.43.4-1.amzn2023.0.4.x86_64.rpm 15 MB/s | 296 kB 00:00
(34/35): xml-common-0.6.3-56.amzn2023.0.2.noarch.rpm 15 MB/s | 32 kB 00:00
(35/35): java-17-amazon-corretto-1:17.0.20-8-1.amzn2023.1.x86_64.rpm 15 MB/s | 220 kB 00:00
Total: 100 MB/s | 100 MB 00:00
Installed: 35 Packages
Total size: 262 MB
Installed size: 262 MB
```

```
ec2-user@ip-10-0-9-174:tmp$ sudo vi /etc/systemd/system/tomcat.service
ec2-user@ip-10-0-9-174:tmp$ sudo systemctl daemon-reload
ec2-user@ip-10-0-9-174:tmp$ sudo systemctl enable tomcat
Created symlink /etc/systemd/system/multi-user.target.wants/tomcat.service → /etc/systemd/system/tomcat.service.
ec2-user@ip-10-0-9-174:tmp$ sudo systemctl start tomcat
ec2-user@ip-10-0-9-174:tmp$ sudo systemctl status tomcat
tomcat.service - Apache Tomcat
Loaded: loaded (/etc/systemd/system/tomcat.service; enabled; preset: disabled)
Active: active (running) since Wed 2026-08-26 14:47:41 UTC; 8s ago
Process: 28460 ExecStart=/opt/tomcat/bin/startup.sh (code=exited, status=0/SUCCESS)
Main PID: 28467 (java)
Tasks: 30 (limit: 1059)
Memory: 75.1M
CPU: 4.144s
CGroup: /system.slice/tomcat.service
└─28467 /usr/lib/jvm/java-17-amazon-corretto/bin/java -Djava.util.logging.config.file=/opt/tomcat/conf/logging.properties -Djava.util.logging.manager=org.apache.juli.ClassLoaderLogManager -Djdk.t...

Aug 26 14:47:41 ip-10-0-9-174.ap-south-1.compute.internal startup.sh[28460]: Starting tomcat.service - Apache Tomcat...
Aug 26 14:47:41 ip-10-0-9-174.ap-south-1.compute.internal startup.sh[28460]: Tomcat started.
Aug 26 14:47:41 ip-10-0-9-174.ap-south-1.compute.internal systemd[1]: Started tomcat.service - Apache Tomcat.
lines 1-14/14 (END)
```

```
[ec2-user@ip-10-0-9-174 tmp]$ sudo ss -lntp | grep 8080
LISTEN 0      100          *:8080      *:      users:(("java",pid=28467,fd=43))
[ec2-user@ip-10-0-9-174 tmp]$ sudo mkdir -p /opt/tomcat/webapps/sample
[ec2-user@ip-10-0-9-174 tmp]$ sudo vi /opt/tomcat/webapps/sample/index.html
[ec2-user@ip-10-0-9-174 tmp]$ sudo chown -R tomcat:tomcat /opt/tomcat/webapps/sample
[ec2-user@ip-10-0-9-174 tmp]$ curl http://localhost:8080/sample/
<html>
<head>
  <title>Sample Application</title>
</head>
<body>
  <h1>Welcome to My Sample Application</h1>
  <p>Tomcat is successfully installed on the private EC2.</p>
</body>
</html>
[ec2-user@ip-10-0-9-174 tmp]$
```

Result: Apache Tomcat was successfully installed and started on the private EC2 instance. A sample web application was deployed under the /sample context and successfully accessed through Tomcat on port **8080**.

10 Configure VPC flow logs and store the logs in S3 and CloudWatch.

Objective

Configure VPC Flow Logs and store network traffic logs in Amazon S3 and CloudWatch Logs.

Steps

1. Created an S3 bucket for VPC Flow Logs.
2. Created a CloudWatch Log Group:
/aws/vpc/DevOps-VPC-FlowLogs
3. Created the IAM role:
VPCFlowLogsCloudWatchRole
4. Created VPC Flow Log:
DevOps-VPC-CloudWatch-FlowLogs
5. Configured the flow log to send traffic logs to CloudWatch Logs.
6. Created VPC Flow Log:
DevOps-VPC-S3-FlowLogs
7. Configured the flow log to store traffic logs in the S3 bucket.
8. Verified both flow logs under the DevOps VPC.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik Asif

Amazon S3

Buckets

Create bucket

General configuration

AWS Region

Asia Pacific (Mumbai) ap-south-1

Bucket type

Info

☒ General purpose

Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ Directory

Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace

Choose the namespace where you want to create your bucket. [Learn more](#)

☒ Global namespace

By default, S3 creates general purpose buckets in the global namespace.

☐ Account Regional namespace (recommended)

General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name

Info

devops-vpc-flow-logs-asif-2026

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

Copy settings from existing bucket - optional

Only the bucket settings in the following configuration are copied.

Choose bucket

Format: s3://bucket/prefix

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik Asif

Amazon S3

Buckets

devops-vpc-flow-logs-asif-2026

Successfully created bucket "devops-vpc-flow-logs-asif-2026"

devops-vpc-flow-logs-asif-2026

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Points

Objects (0)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name

Type

Last modified

Size

Storage class

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik Asif

CloudWatch

Log management

Create log group

CloudWatch

<

Favorites and recents

►

Ingestion

Dashboards

Alarms

► AI Operations

► GenAI Observability

► Application Signals (APM)

► Infrastructure Monitoring

▼ Logs

Log Management

Log Analytics

Log Anomalies

► Metrics

► Network Monitoring

Create log group

Log group details

Info

☒ CloudWatch Logs offers three log classes: Standard, Infrequent Access, and Delivery. [Learn more about the features offered by each log class.](#)

Log group name

/aws/vpc/DevOps-VPC-FlowLogs

Retention setting

Never expire

Log class

Info

Standard

KMS key ARN - optional

Choose an AWS KMS key or enter an ARN

Create an AWS KMS key

☐ Deletion protection

Deletion protection is turned off by default. Deletion protection helps prevent accidental or unintended deletion of log groups that store critical operational or compliance data.

CloudWatch > Log management

Log group `/aws/vpc/DevOps-VPC-FlowLogs*` has been created.

Log groups | Data sources - new | Summary

Get-set-go with unified data store

Log groups (1)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

Exact match

Log group	Log class	Anomaly d...	Deleti...	Bearer...	Dat...	Sen...	Retenti...
/aws/vpc/DevOps-VPC-FlowLogs	Standard	Configure	Off	Off	-	-	Never exp...

IAM > Roles

Role `VPCFlowLogsCloudWatchRole` created.

Roles (6) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

Role name	Trusted entities	Last activity
AWSServiceRoleForEC2Spot	AWS Service: spot (Service-Linked Role)	-
AWSServiceRoleForNATGateway	AWS Service: ec2-nat-gateway (Service-Linked Role)	2 hours ago
AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Service-Linked Role)	12 minutes ago
AWSServiceRoleForSupport	AWS Service: support (Service-Linked Role)	-
AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linked Role)	-
VPCFlowLogsCloudWatchRole	AWS Service: ec2	-

Roles Anywhere Info

Authenticate your non AWS workloads and securely provide access to AWS services.

Manage

aws

Search

[Alt+S]

Global

Shaik Asif (331116047691)

Shaik Asif

IAM

Roles

VPCFlowLogsCloudWatchRole

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Policy simulator

VPCFlowLogsCloudWatchRole

Info

Delete

Allows EC2 instances to call AWS services on your behalf.

Summary

Edit

Creation date

August 26, 2026, 20:37 (UTC+05:30)

ARN

arn:aws:iam::331116047691:role/VPCFlowLogsCloudWatchRole

Instance profile ARN

arn:aws:iam::331116047691:instance-profile/VPCFlowLogsCloudWatchRole

Last activity

-

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (1)

Info

Simulate

Remove

Add permissions

You can attach up to 20 managed policies.

Search

Filter by Type

All types

< 1 >

☐

Policy name

▲

Type

▼

Attached entities

▼

☐

CloudWatchLogsFullAccess

AWS managed

1

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik Asif

VPC

Your VPCs

Create flow logs

Create flow log

Info

Flow logs can capture IP traffic flow information for the network interfaces associated with your resources. You can create multiple flow logs to send traffic to different destinations.

Selected resources

Info

Flow logs will only be created for resources in an available state.

Name

▼

Resource ID

▼

State

▼

DevOps-VPC

vpc-02e579ff687daa21e

Available

Flow log settings

Name - optional

DevOps-VPC-CloudWatch-FlowLogs

Filter

The type of traffic to capture (accepted traffic only, rejected traffic only, or all traffic).

☐ Accept

☐ Reject

☒ All

Maximum aggregation interval

Info

The maximum interval of time during which a flow of packets is captured and aggregated into a flow log record.

☐ 10 minutes

☒ 1 minute

Destination

The destination to which to publish the flow log data.

☒ Send to CloudWatch Logs

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik Asif

VPC

>

Your VPCs

>

fl-0ac95ffe2a490a88a

VPC dashboard

<

Filter by VPC

AWS Global View

Virtual private cloud

>

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

>

Network ACLs

Security groups

fl-0ac95ffe2a490a88a / DevOps-VPC-CloudWatch-FlowLogs

Actions

Details

Flow Log ID

fl-0ac95ffe2a490a88a

Destination Type

cloud-watch-logs

Traffic Type

All

File Format

-

Name

DevOps-VPC-CloudWatch-FlowLogs

Destination Name

/aws/vpc/DevOps-VPC-FlowLogs

Max Aggregation Interval

1 minute

Hive Compatible Partitions

-

State

Active

IAM Role

arn:aws:iam::331116047691:role/VPCFlowLogsCloudWatchRole

Log Format

Default

Partition Logs

-

Creation Time

Wednesday, August 26, 2026 at 20:41:37 GMT+5:30

Cross Account IAM Role

-

Tag field specifications

-

Tags

Integrations

Tags

Search tags

Manage tags

< 1 >

Key

Value

Name

DevOps-VPC-CloudWatch-FlowLogs

CloudShell

Agent Toolkit for AWS

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

VPC

>

Your VPCs

>

Create flow logs

Selected resources

Info

Flow logs will only be created for resources in an available state.

Name

Resource ID

State

DevOps-VPC

vpc-02e579ff687daa21e

Available

Flow log settings

Name - optional

DevOps-VPC-S3-FlowLogs

Filter

The type of traffic to capture (accepted traffic only, rejected traffic only, or all traffic).

Accept

Reject

All

Maximum aggregation interval

Info

The maximum interval of time during which a flow of packets is captured and aggregated into a flow log record.

10 minutes

1 minute

Destination

The destination to which to publish the flow log data.

Send to CloudWatch Logs

Send to an Amazon S3 bucket

Send to Amazon Data Firehose in the same account

Send to Amazon Data Firehose in a different account

fl-0dd00e4d4bfff55f2 / DevOps-VPC-S3-FlowLogs

Details

Flow Log ID fl-0dd00e4d4bfff55f2	Destination Type s3	Traffic Type All	File Format Plain text
Name DevOps-VPC-S3-FlowLogs	Destination Name devops-vpc-flow-logs-asif-2026	Max Aggregation Interval 1 minute	Hive Compatible Partitions Not enabled
State Active	IAM Role -	Log Format Default	Partition Logs Daily
Creation Time Wednesday, August 26, 2026 at 20:44:41 GMT+5:30	Cross Account IAM Role -	Tag field specifications -	

Tags

Key	Value
Name	DevOps-VPC-S3-FlowLogs

Your VPCs

Your VPCs (1/2)

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv...
-	vpc-029ed55172cb55c8d	Available	-	-	Off	172
DevOps-VPC	vpc-02e579ff687daa21e	Available	-	-	Off	10.

vpc-02e579ff687daa21e / DevOps-VPC

Flow logs (2)

Name	Flow log ID	Traffic type	Destination type
DevOps-VPC-S3-FlowLogs	fl-0dd00e4d4bfff55f2	All	s3
DevOps-VPC-CloudWatch-FlowLogs	fl-0ac95ffe2a490a88a	All	cloud-watch-logs

Result

VPC Flow Logs were successfully configured for the DevOps VPC. One flow log was configured to send network traffic logs to CloudWatch Logs, and another was configured to store the logs in an S3 bucket. Both flow log configurations were successfully created and verified.